WIKIPEDIA

Cilk

From Wikipedia, the free encyclopedia

Cilk, **Cilk++**, **Cilk Plus** and **OpenCilk** are general-purpose programming languages designed for multithreaded parallel computing. They are based on the C and C++ programming languages, which they extend with constructs to express parallel loops and the fork–join idiom.

Originally developed in the 1990s at the Massachusetts Institute of Technology (MIT) in the group of Charles E. Leiserson, Cilk was later commercialized as Cilk++ by a spinoff company, Cilk Arts. That company was subsequently acquired by Intel, which increased compatibility with existing C and C++ code, calling the result Cilk Plus. After Intel stopped supporting Cilk Plus in 2017, MIT is again developing Cilk in the form of OpenCilk.

Contents

Cilk	
Paradigm	imperative (procedural), structured, parallel
Designed by	MIT Laboratory for Computer Science
Developer	Intel
First appeared	1994
Typing discipline	static, weak, manifest
Website	cilk.mit.edu (https://cilk.mit.edu/)
Dialects	
Cilk++, Cilk Plus, OpenCilk	
Influenced by	
C	
Influenced	
OpenMP 3.0,[1] Rayon (Rust library)[2]	

OpenCilk	
Designed by	MIT
Developer	MIT
First appeared	2020
Stable release	2.0.1 / September 3, 2022
OS	Unix-like, macOS
License	MIT
Website	www.opencilk.org (https://www.opencilk.org)

Cilk Plus	
Designed by	Intel
Developer	Intel
First appeared	2010

History

MIT Cilk

Stable release	1.2 / September 9, 2013
Filename extensions	(Same as C or C++)
Website	http://cilkplus.org/ (https://web.archive.org/web/20210117031010/http://cilkplus.org/)

The Cilk programming language grew out of three separate projects at the MIT Laboratory for Computer Science:[3]

- Theoretical work on scheduling multithreaded applications.
- StarTech – a parallel chess program built to run on the Thinking Machines Corporation's Connection Machine model CM-5.
- PCM/Threaded-C – a C-based package for scheduling continuation-passing-style threads on the CM-5

In April 1994 the three projects were combined and christened "Cilk". The name Cilk is not an acronym, but an allusion to "nice threads" (silk) and the C programming language. The Cilk-1 compiler was released in September 1994.

The original Cilk language was based on ANSI C, with the addition of Cilk-specific keywords to signal parallelism. When the Cilk keywords are removed from Cilk source code, the result should always be a valid C program, called the *serial elision* (or *C elision*) of the full Cilk program, with the same semantics as the Cilk program running on a single processor. Despite several similarities, Cilk is not directly related to AT&T Bell Labs' Concurrent C.

Cilk was implemented as a translator to C, targeting the GNU C Compiler (GCC). The last version, Cilk 5.4.6, is available from the MIT Computer Science and Artificial Intelligence Laboratory (CSAIL), but is no longer supported.[4]

A showcase for Cilk's capabilities was the Cilkchess parallel chess-playing program, which won several computer chess prizes in the 1990s, including the 1996 Open Dutch Computer Chess Championship.[5]

Cilk Arts and Cilk++

Prior to c. 2006, the market for Cilk was restricted to high-performance computing. The emergence of multicore processors in mainstream computing meant that hundreds of millions of new parallel computers were being shipped every year. Cilk Arts was formed to capitalize on that opportunity: in 2006, Leiserson launched Cilk Arts to create and bring to market a modern version of Cilk that supports the commercial needs of an upcoming generation of programmers. The company closed a Series A venture financing round in October 2007, and its product, Cilk++ 1.0, shipped in December, 2008.

Cilk++ differs from Cilk in several ways: support for C++, support for loops, and hyperobjects – a new construct designed to solve data race problems created by parallel accesses to global variables. Cilk++ was proprietary software. Like its predecessor, it was implemented as a Cilk-to-C++ compiler. It supported the Microsoft and GNU compilers.

Intel Cilk Plus

On July 31, 2009, Cilk Arts announced on its web site that its products and engineering team were now part of Intel Corp. In early 2010, the Cilk website at `www.cilk.com` began redirecting to the Intel website (as of early 2017, the original Cilk website no longer resolves to a host). Intel and Cilk Arts integrated and advanced the technology further resulting in a September 2010 release of Intel Cilk Plus.[6][7] Cilk Plus adopts simplifications, proposed by Cilk Arts in Cilk++, to eliminate the need for several of the original Cilk keywords while adding the ability to spawn functions and to deal with variables involved in reduction operations. Cilk Plus differs from Cilk and Cilk++ by adding array extensions, being incorporated in a commercial compiler (from Intel), and compatibility with existing debuggers.[8]

Cilk Plus was first implemented in the Intel C++ Compiler with the release of the Intel compiler in Intel Composer XE 2010. An open source (BSD-licensed) implementation was contributed by Intel to the GNU Compiler Collection (GCC), which shipped Cilk Plus support in version 4.9,[9] except for the `_Cilk_for` keyword, which was added in GCC 5.0. In February 2013, Intel announced a Clang fork with Cilk Plus support.[10] The Intel Compiler, but not the open source implementations, comes with a race detector and a performance analyzer.

Intel later discontinued it, recommending its users switch to instead using either OpenMP or Intel's own TBB library for their parallel programming needs.[11]

Differences between versions

In the original MIT Cilk implementation, the first Cilk keyword is in fact `cilk`, which identifies a function which is written in Cilk. Since Cilk procedures can call C procedures directly, but C procedures cannot directly call or spawn Cilk procedures, this keyword is needed to distinguish Cilk code from C code. Cilk Plus removes this restriction, as well as the `cilk` keyword, so C and C++ functions can call into Cilk Plus code and vice versa.

Deprecation of Cilk Plus

In May, 2017, GCC 7.1 was released and marked Cilk Plus support as deprecated.[12] Intel itself announced in September 2017 that they would deprecate Cilk Plus with the 2018 release of the Intel Software Development Tools.[11] In May 2018, GCC 8.1 was released with Cilk Plus support removed.[13]

OpenCilk

After Cilk Plus support was deprecated by Intel, MIT has taken on the development of Cilk in the OpenCilk implementation, focusing on the LLVM/Clang fork now termed "Tapir".[11][14] OpenCilk remains largely compatible with Intel Cilk Plus.[15] Its first stable version was released in March 2021.[16]

Language features

The principle behind the design of the Cilk language is that the programmer should be responsible for *exposing* the parallelism, identifying elements that can safely be executed in parallel; it should then be left to the run-time environment, particularly the scheduler, to decide during execution how to actually divide the work between processors. It is because these responsibilities are separated that a Cilk program can

run without rewriting on any number of processors, including one.

Task parallelism: spawn and sync

Cilk's main addition to C are two keywords that together allow writing task-parallel programs.

- The `spawn` keyword, when preceding a function call (`spawn f(x)`), indicates that the function call (`f(x)`) can safely run in parallel with the statements following it in the calling function. Note that the scheduler is not *obligated* to run this procedure in parallel; the keyword merely alerts the scheduler that it can do so.
- A `sync` statement indicates that execution of the current function cannot proceed until all previously spawned function calls have completed. This is an example of a barrier method.

(In Cilk Plus, the keywords are spelled `_Cilk_spawn` and `_Cilk_sync`, or `cilk_spawn` and `cilk_sync` if the Cilk Plus headers are included.)

Below is a recursive implementation of the Fibonacci function in Cilk, with parallel recursive calls, which demonstrates the `spawn`, and `sync` keywords. The original Cilk required any function using these to be annotated with the `cilk` keyword, which is gone as of Cilk Plus. (Cilk program code is not numbered; the numbers have been added only to make the discussion easier to follow.)

```
 1  cilk int fib(int n) {
 2      if (n < 2) {
 3          return n;
 4      }
 5      else {
 6         int x, y;
 7
 8         x = spawn fib(n - 1);
 9         y = spawn fib(n - 2);
10
11         sync;
12
13         return x + y;
14      }
15  }
```

If this code was executed by a *single* processor to determine the value of `fib(2)`, that processor would create a frame for `fib(2)`, and execute lines 1 through 5. On line 6, it would create spaces in the frame to hold the values of `x` and `y`. On line 8, the processor would have to suspend the current frame, create a new frame to execute the procedure `fib(1)`, execute the code of that frame until reaching a return statement, and then resume the `fib(2)` frame with the value of fib(1) placed into `fib(2)`'s `x` variable. On the next line, it would need to suspend again to execute `fib(0)` and place the result in `fib(2)`'s `y` variable.

When the code is executed on a *multiprocessor* machine, however, execution proceeds differently. One processor starts the execution of `fib(2)`; when it reaches line 8, however, the `spawn` keyword modifying the call to `fib(n-1)` tells the processor that it can safely give the job to a second processor: this second processor can create a frame for `fib(1)`, execute its code, and store its result in `fib(2)`'s frame when it finishes; the first processor continues executing the code of `fib(2)` at the same time. A processor is not obligated to assign a spawned procedure elsewhere; if the machine only has two processors and the second is still busy on `fib(1)` when the

processor executing `fib(2)` gets to the procedure call, the first processor will suspend `fib(2)` and execute `fib(0)` itself, as it would if it were the only processor. Of course, if another processor is available, then it will be called into service, and all three processors would be executing separate frames simultaneously.

(The preceding description is not entirely accurate. Even though the common terminology for discussing Cilk refers to processors making the decision to spawn off work to other processors, it is actually the scheduler which assigns procedures to processors for execution, using a policy called *work-stealing*, described later.)

If the processor executing `fib(2)` were to execute line 13 before both of the other processors had completed their frames, it would generate an incorrect result or an error; `fib(2)` would be trying to add the values stored in `x` and `y`, but one or both of those values would be missing. This is the purpose of the `sync` keyword, which we see in line 11: it tells the processor executing a frame that it must suspend its own execution until all the procedure calls it has spawned off have returned. When `fib(2)` is allowed to proceed past the `sync` statement in line 11, it can only be because `fib(1)` and `fib(0)` have completed and placed their results in `x` and `y`, making it safe to perform calculations on those results.

The code example above uses the syntax of Cilk-5. The original Cilk (Cilk-1) used a rather different syntax that required programming in an explicit continuation-passing style, and the Fibonacci examples looks as follows:[17]

```
thread fib(cont int k, int n)
{
    if (n < 2) {
        send_argument(k, n);
    }
    else {
        cont int x, y;
        spawn_next sum(k, ?x, ?y);
        spawn fib(x, n - 1);
        spawn fib(y, n - 2);
    }
}

thread sum(cont int k, int x, int y)
{
     send_argument(k, x + y);
}
```

Inside `fib`'s recursive case, the `spawn_next` keyword indicates the creation of a *successor* thread (as opposed to the *child* threads created by `spawn`), which executes the `sum` subroutine after waiting for the *continuation variables* `x` and `y` to be filled in by the recursive calls. The base case and `sum` use a `send_argument(k, n)` operation to set their continuation variable `k` to the value of `n`, effectively "returning" the value to the successor thread.

Inlets

The two remaining Cilk keywords are slightly more advanced, and concern the use of *inlets*. Ordinarily, when a Cilk procedure is spawned, it can return its results to the parent procedure only by putting those results in a variable in the parent's frame, as we assigned the results of our spawned procedure calls in the example to `x` and `y`.

The alternative is to use an inlet. An inlet is a function internal to a Cilk procedure which handles the results of a spawned procedure call as they return. One major

reason to use inlets is that all the inlets of a procedure are guaranteed to operate atomically with regards to each other and to the parent procedure, thus avoiding the bugs that could occur if the multiple returning procedures tried to update the same variables in the parent frame at the same time.

- The `inlet` keyword identifies a function defined within the procedure as an inlet.
- The `abort` keyword can only be used inside an inlet; it tells the scheduler that any other procedures that have been spawned off by the parent procedure can safely be aborted.

Inlets were removed when Cilk became Cilk++, and are not present in Cilk Plus.

Parallel loops

Cilk++ added an additional construct, the parallel loop, denoted `cilk_for` in Cilk Plus. These loops look like

```
1  void loop(int *a, int n)
2  {
3      #pragma cilk grainsize = 100  // optional
4      cilk_for (int i = 0; i < n; i++) {
5          a[i] = f(a[i]);
6      }
7  }
```

This implements the parallel map idiom: the body of the loop, here a call to `f` followed by an assignment to the array `a`, is executed for each value of `i` from zero to `n` in an indeterminate order. The optional "grain size" pragma determines the coarsening: any sub-array of one hundred or fewer elements is processed sequentially. Although the Cilk specification does not specify the exact behavior of the construct, the typical implementation is a divide-and-conquer recursion,[18] as if the programmer had written

```
static void recursion(int *a, int start, int end)
{
    if (end - start <= 100) {  // The 100 here is the grainsize.
        for (int i = start; i < end; i++) {
            a[i] = f(a[i]);
        }
    }
    else {
        int midpoint = start + (end - start) / 2;
        cilk_spawn recursion(a, start, midpoint);
        recursion(a, midpoint, end);
        cilk_sync;
    }
}

void loop(int *a, int n)
{
    recursion(a, 0, n);
}
```

The reasons for generating a divide-and-conquer program rather than the obvious alternative, a loop that spawn-calls the loop body as a function, lie in both the grainsize handling and in efficiency: doing all the spawning in a single task makes load balancing a bottleneck.[19]

A review of various parallel loop constructs on HPCwire found the `cilk_for` construct to be quite general, but noted that the Cilk Plus specification did not stipulate that its iterations need to be data-independent, so a compiler cannot automatically vectorize a `cilk_for` loop. The review also noted the fact that reductions (e.g., sums over arrays) need additional code.[18]

Reducers and hyperobjects

Cilk++ added a kind of objects called *hyperobjects*, that allow multiple strands to share state without race conditions and without using explicit locks. Each strand has a view on the hyperobject that it can use and update; when the strands synchronize, the views are combined in a way specified by the programmer.[20]

The most common type of hyperobject is a reducer, which corresponds to the reduction clause in OpenMP or to the algebraic notion of a monoid. Each reducer has an identity element and an associative operation that combines two values. The archetypal reducer is summation of numbers: the identity element is zero, and the associative *reduce* operation computes a sum. This reducer is built into Cilk++ and Cilk Plus:

```
// Compute ∑ foo(i) for i from 0 to N, in parallel.
cilk::reducer_opadd<float> result(0);
cilk_for (int i = 0; i < N; i++)
    result += foo(i);
```

Other reducers can be used to construct linked lists or strings, and programmers can define custom reducers.

A limitation of hyperobjects is that they provide only limited determinacy. Burckhardt *et al.* point out that even the sum reducer can result in non-deterministic behavior, showing a program that may produce either 1 or 2 depending on the scheduling order:[21]

```
void add1(cilk::reducer_opadd<int> &r) { r++; }
// ...
cilk::reducer_opadd<int> r(0);
cilk_spawn add1(r);
if (r == 0) { r++; }
cilk_sync;
output(r.get_value());
```

Array notation

Intel Cilk Plus adds notation to express high-level operations on entire arrays or sections of arrays; e.g., an axpy-style function that is ordinarily written

```
// y ← α x + y
void axpy(int n, float alpha, const float *x, float *y)
{
    for (int i = 0; i < n; i++) {
        y[i] += alpha * x[i];
    }
}
```

can in Cilk Plus be expressed as

```
y[0:n] += alpha * x[0:n];
```

This notation helps the compiler to effectively vectorize the application. Intel Cilk Plus allows C/C++ operations to be applied to multiple array elements in parallel, and also provides a set of built-in functions that can be used to perform vectorized shifts, rotates, and reductions. Similar functionality exists in Fortran 90; Cilk Plus differs in that it never allocates temporary arrays, so memory usage is easier to predict.

Elemental functions

In Cilk Plus, an elemental function is a regular function which can be invoked either on scalar arguments or on array elements in parallel. They are similar to the kernel functions of OpenCL.

#pragma simd

This pragma gives the compiler permission to vectorize a loop even in cases where auto-vectorization might fail. It is the simplest way to manually apply vectorization.

Work-stealing

The Cilk scheduler uses a policy called "work-stealing" to divide procedure execution efficiently among multiple processors. Again, it is easiest to understand if we look first at how Cilk code is executed on a single-processor machine.

The processor maintains a stack on which it places each frame that it has to suspend in order to handle a procedure call. If it is executing *fib(2)*, and encounters a recursive call to *fib(1)*, it will save *fib(2)*'s state, including its variables and where the code suspended execution, and put that state on the stack. It will not take a suspended state off the stack and resume execution until the procedure call that caused the suspension, and any procedures called in turn by that procedure, have all been fully executed.

With multiple processors, things of course change. Each processor still has a stack for storing frames whose execution has been suspended; however, these stacks are more like deques, in that suspended states can be removed from either end. A processor can still only remove states from its *own* stack from the same end that it puts them on; however, any processor which is not currently working (having finished its own work, or not yet having been assigned any) will pick another processor at random, through the scheduler, and try to "steal" work from the opposite end of their stack – suspended states, which the stealing processor can then begin to execute. The states which get stolen are the states that the processor stolen from would get around to executing last.

See also

- Grand Central Dispatch
- Intel Concurrent Collections (CnC)
- Intel Parallel Building Blocks (PBB)
  - Intel Array Building Blocks (ArBB)

- Intel Parallel Studio
- NESL
- OpenMP
- Parallel computing
- Sieve C++ Parallel Programming System
- Threading Building Blocks (TBB)
- Unified Parallel C

References

1. LaGrone, James; Aribuki, Ayodunni; Addison, Cody; Chapman, Barbara (2011). *A Runtime Implementation of OpenMP Tasks*. 7th Int'l Workshop on OpenMP. pp. 165–178. CiteSeerX 10.1.1.221.2775 (https://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.221.2775). doi:10.1007/978-3-642-21487-5_13 (https://doi.org/10.1007%2F978-3-642-21487-5_13).
2. "Rayon FAQ" (https://github.com/rayon-rs/rayon/blob/master/FAQ.md#how-does-rayon-balance-work-between-threads). *GitHub*. "The name rayon is a homage to that work."
3. " "A Brief History of Cilk" (https://web.archive.org/web/20150626125629/https://www.cilkplus.org/cilk-history). Archived from the original (https://www.cilkplus.org/cilk-history) on 2015-06-26. Retrieved 2015-06-25.
4. "The Cilk Project" (http://supertech.csail.mit.edu/cilk/). MIT CSAIL. 8 October 2010. Retrieved 25 January 2016.
5. Leiserson, Charles E.; Plaat, Aske (1998). "Programming parallel applications in Cilk" (https://www.researchgate.net/publication/2427921). *SIAM News*. **31**.
6. "Intel Flexes Parallel Programming Muscles" (http://www.hpcwire.com/features/Intel-Flexes-Parallel-Programming-Muscles-102084438.html) Archived (https://web.archive.org/web/20100906030803/http://www.hpcwire.com/features/Intel-Flexes-Parallel-Programming-Muscles-102084438.html) 2010-09-06 at the Wayback Machine, HPCwire (2010-09-02). Retrieved on 2010-09-14.
7. "Parallel Studio 2011: Now We Know What Happened to Ct, Cilk++, and RapidMind" (http://www.drdobbs.com/go-parallel/blog/archives/2010/09/parallel_studio_1.html) Archived (https://web.archive.org/web/20100926110843/http://www.drdobbs.com/go-parallel/blog/archives/2010/09/parallel_studio_1.html) 2010-09-26 at the Wayback Machine, Dr. Dobb's Journal (2010-09-02). Retrieved on 2010-09-14.
8. "Intel Cilk Plus: A quick, easy and reliable way to improve threaded performance" (http://software.intel.com/en-us/articles/intel-cilk-plus/), Intel. Retrieved on 2010-09-14.
9. "GCC 4.9 Release Series Changes, New Features, and Fixes" (https://gcc.gnu.org/gcc-4.9/changes.html), Free Software Foundation, Inc. Retrieved on 2014-06-29.
10. Cilk Plus/LLVM (https://cilkplus.github.io/)
11. Hansang B. (20 September 2017). "Intel Cilk Plus is being deprecated" (https://software.intel.com/en-us/forums/intel-cilk-plus/topic/745556). *Intel Cilk Plus forum*.
12. "GCC 7 Release Series. Changes, New Features, and Fixes" (https://gcc.gnu.org/gcc-7/changes.html). *GCC, the GNU Compiler Collection*.
13. "GCC 8 Release Series. Changes, New Features, and Fixes" (https://gcc.gnu.org/gcc-8/changes.html). *GCC, the GNU Compiler Collection*.

14. "Cilk Hub taking on Cilk development after Intel announcement" (http://cilk.mit.edu/cilkhub/2017/12/01/cilkhub/). *OpenCilk*. 2017-12-01. Archived (https://web.archive.org/web/20180612204746/http://cilk.mit.edu/cilkhub/2017/12/01/cilkhub/) from the original on 2018-06-12. Retrieved 2021-12-06.
15. "OpenCilk" (https://cilk.mit.edu/). *OpenCilk*. Retrieved 2021-12-06.
16. "Release opencilk/v1.0 · OpenCilk/opencilk-project" (https://github.com/OpenCilk/opencilk-project/releases/tag/opencilk%2Fv1.0). *GitHub*. 2021-03-05. Archived (https://web.archive.org/web/20211206122729/https://github.com/OpenCilk/opencilk-project/releases/tag/opencilk%2Fv1.0) from the original on 2021-12-06. Retrieved 2021-12-06.
17. Blumofe, Robert D.; Joerg, Christopher F.; Kuszmaul, Bradley C.; Leiserson, Charles E.; Randall, Keith H.; Zhou, Yuli (1995). *Cilk: An Efficient Multithreaded Runtime System* (http://supertech.csail.mit.edu/papers/PPoPP95.pdf) (PDF). Proc. ACM SIGPLAN Symp. Principles and Practice of Parallel Programming. pp. 207–216.
18. Wolfe, Michael (6 April 2015). "Compilers and More: The Past, Present and Future of Parallel Loops" (http://www.hpcwire.com/2015/04/06/compilers-and-more-the-past-present-and-future-of-parallel-loops/). *HPCwire*.
19. McCool, Michael; Reinders, James; Robison, Arch (2013). *Structured Parallel Programming: Patterns for Efficient Computation*. Elsevier. p. 30.
20. Frigo, Matteo; Halpern, Pablo; Leiserson, Charles E.; Lewin-Berlin, Stephen (2009). *Reducers and other Cilk++ hyperobjects* (http://www.fftw.org/~athena/papers/hyper.pdf) (PDF). Proc. Annual Symposium on Parallelism in Algorithms and Architectures (SPAA). ACM.
21. Burckhardt, Sebastian; Baldassin, Alexandro; Leijen, Daan (2010). *Concurrent Programming with Revisions and Isolation Types* (http://research.microsoft.com/pubs/132619/revisions-oopsla2010.pdf) (PDF). Proc. OOPSLA/SPLASH.

External links

- Official website (http://cilk.mit.edu/) for OpenCilk
- Intel's Cilk Plus website (https://web.archive.org/web/20210117031010/http://cilkplus.org/)
- Cilk Project website at MIT (https://web.archive.org/web/20201128222246/http://supertech.csail.mit.edu/cilk/)
- Arch D. Robison, "Cilk Plus: Language Support for Thread and Vector Parallelism" (http://parallelbook.com/sites/parallelbook.com/files/CilkPlus-HPCAST18_0.pdf) and "Parallel Programming with Cilk Plus" (http://www.parallelbook.com/sites/parallelbook.com/files/ISC2012_Tutorial_9_CilkPlus_Robison_final.pdf), July 16, 2012.